

challenge_reverseeng

CHALLENGE

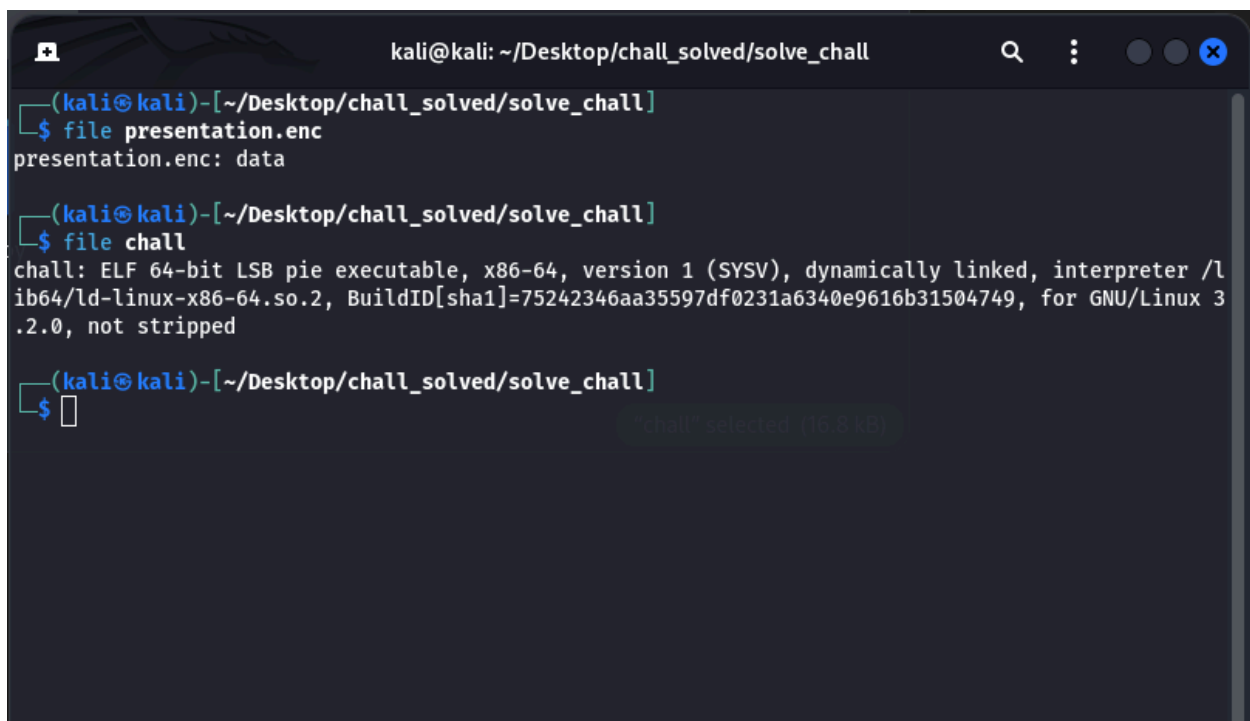
f had zip kayn 2 files, "chall" li how binary li khaskom diro 3lih REV w 3ndkom "presentation.enc" li hiya presentation encrypted.

goal dyalkom how t3rfo kifach encryptina l presentation bach diro liha decrypt w tl9aw l presentation.

good luck :)

The approach that i used

running file



```
kali@kali: ~/Desktop/chall_solved/solve_chall
(kali@kali)-[~/Desktop/chall_solved/solve_chall]
$ file presentation.enc
presentation.enc: data

(kali@kali)-[~/Desktop/chall_solved/solve_chall]
$ file chall
chall: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=75242346aa35597df0231a6340e9616b31504749, for GNU/Linux 3.2.0, not stripped

(kali@kali)-[~/Desktop/chall_solved/solve_chall]
$
```

A **non-stripped binary** keeps function names (like `main`, `printf`, or even custom functions) and sometimes variable names. This is a huge advantage for static

analysis tools because it allows you to understand the flow of the program more quickly.

Because the binary is **not stripped**, I used **Ghidra**

⇒ Ghidra takes compiled code (like ELF) and allows you to **analyze and understand the program without source code**.

Since the ELF binary is not stripped, it retains function and symbol names, which allows Ghidra to produce a more readable decompiled output. This makes understanding the program's logic and identifying the vulnerability much easier than analyzing a stripped binary.

main function

```
undefined8 main(int argc,undefined8 *argv)

{
    undefined8 uVar1;
    size_t local_38;
    FILE *local_30;
    char *key;
    void *file;
    size_t size;
    FILE *file_src;

    if (argc == 3) {
        file_src = fopen((char *)argv[1],"rb");
        if (file_src == (FILE *)0x0) {
            puts("Error: Cannot open input file");
            uVar1 = 1;
        }
        else {
            fseek(file_src,0,2);
            size = ftell(file_src);
            fseek(file_src,0,0);
```

```

file = malloc(size);
fread(file,1,size,file_src);
fclose(file_src);
key = "wa_db_chof_ki_dir_w_kmal_steps_lakhrin";
printf("[*] Loading PDF file (%zu bytes)...\n",size);
puts("[*] Stage 1: XOR encryption with rotating key...");
xor_encrypt((long)file,size,key);
puts("[*] Stage 2: Byte substitution using S-box...");
substitute_bytes((long)file,size);
puts("[*] Stage 3: Bit rotation (5 bits left)...");
rotate_bits((long)file,size,5);
puts("[*] Stage 4: Byte interleaving...");
interleave_bytes(file,size);
local_30 = fopen((char *)argv[2],"wb");
if (local_30 == (FILE *)0x0) {
    puts("Error: Cannot create output file");
    free(file);
    uVar1 = 1;
}
else {
    fwrite("PDFENC",1,6,local_30);
    local_38 = size;
    fwrite(&local_38,1,8,local_30);
    fwrite(file,1,size,local_30);
    fclose(local_30);
    free(file);
    puts("[+] Challenge file created successfully!");
    printf("[+] Output: %s\n",(char *)argv[2]);
    puts("[+] Good luck reversing this! ;)");
    uVar1 = 0;
}
}
}
else {
    printf("Usage: %s <input_pdf> <output_file>\n",(char *)*argv);
    puts("Challenge: Reverse engineer this program to recover the original PD

```

```

F!");
    uVar1 = 1;
}
return uVar1;
}

```

noticed that the code takes 2 path's the file that we wants to encrypt (src_file)and the destination

the program takes the file src_file we open it and read binary data from it

then we start encrypting using xor_encrypt takes 3 params file, size of binary data, key

```

void xor_encrypt(long file,ulong size,char *key)

{
    size_t sVar1;
    ulong i;

    sVar1 = strlen(key);
    for (i = 0; i < size; i = i + 1) {
        *(byte *)(i + file) = *(byte *)(i + file) ^ key[i % sVar1] ^ (byte)i;
    }
    return;
}
//the code here iterate on the file each char
//and xor it with key[with random value (from 0 to sizeof key)]
//then xor it with variable i then assign it to file[i]
/* Example:
Original data: Hello, World!
Encrypted data (hex): 48 0F 3F 2D 0B 6C 7A 5B 21 6D 2A 4F 6F
*/

```

then we take xor output and give it to substitute_bytes function that takes 2 params

```

void substitute_bytes(long file,ulong size)

{
    ulong i;

    for (i = 0; i < size; i = i + 1) {
        *(undefined1*)(i + file) = sbox.0[(int)(uint)*(byte*)(i + file)];
    }
    return;
}
//iterate on file take the char and cast it to int
//so we chose from the sbox a rand value
//sbox its an array that has 0-255 random values
//assign the random value to the file[i]
/* Example: (IT DEPENDS ON SBOX)
data : 48 0F 3F 2D 0B 6C 7A 5B 21 6D 2A 4F 6F
Substituted data (hex): B7 F0 C0 D2 F4 93 85 A4 DE 92 D5 B0 90

```

then rotate_bits takes the output of the function before and the size and number of rotation its 5 here

```

void rotate_bits(long file,ulong size,undefined4 5)

{
    byte bVar1;
    undefined8 i;

    for (i = 0; i < size; i = i + 1) {
        bVar1 = (byte)((int)5 >> 0x1f);
        *(byte*)(i + file) =
            *(char*)(i + file) << ((byte)5 & 7) |
            (byte)((int)(uint)*(byte*)(i + file) >>
                (8 - (((byte)5 + (bVar1 >> 5) & 7) - (bVar1 >> 5)) & 0x1f));
    }
    return;
}

```

```

}
// Ai explanation:
/*Step by Step an example using "A"
Original byte: 0100 0001
OR them together: 0000 1000 | 0000 0100 = 0010 0001

```

then the final function

```

void interleave_bytes(void *file,size_t size)

{
    void *__dest;
    undefined8 local_10;

    __dest = malloc(size);
    memcpy(__dest,file,size);
    for (local_10 = 0; local_10 < size >> 1; local_10 = local_10 + 1) {
        *(undefined1 *)((long)file + local_10 * 2) = *(undefined1 *)((local_10 + (long)_
__dest);
        *(undefined1 *)((long)file + local_10 * 2 + 1) =
            *(undefined1 *)((long)__dest + (size >> 1) + local_10);
    }
    if ((size & 1) != 0) {
        *(undefined1 *)((long)file + (size - 1)) = *(undefined1 *)((long)__dest + (size
- 1));
    }
    free(__dest);
    return;
}
/*
Split the buffer into two halves
file[0] = first_half[0]
file[1] = second_half[0]
file[2] = first_half[1]
file[3] = second_half[1]

```

... etc

If the size is odd, copy the last byte from the original to the end

and lastely it adds to the file "PDFESC"



by running :

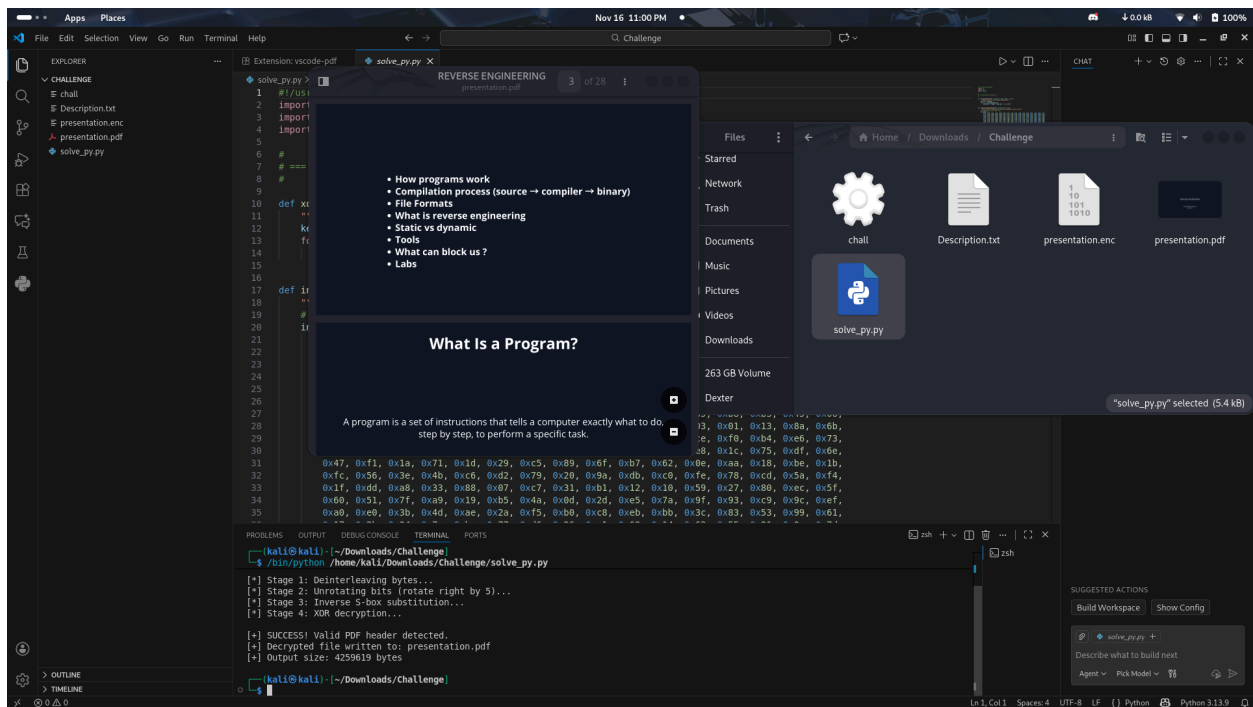
hexfile presentation.enc

```
kali@kali: ~/Downloads/Challenge
00000000  50 44 46 45 4E 43 23 FF 40 00 00 00 00 00 00 D6 PDFENC#. @.....
00000010  80 79 9A FF BF 1A 76 31 EF 93 76 44 27 66 87 6C .y....v1..vD'f.l
00000020  DA E1 41 27 EB 25 DD 4E AB 76 1A AC 6B 9F 85 77 ..A'%.N.v..k..w
00000030  7E D5 CD 05 C4 B6 94 B4 39 4E FF 3D 14 DF 24 DD ~.....9N.=...$.
00000040  1A 5E 70 08 BC 0C 2B 1C 4E 7D C6 05 BD 97 F3 01 .^p...+.N}.....
00000050  20 86 3E 69 E7 D6 2E F8 99 BD 56 AF BF CA FF 00 .>i.....V.....
00000060  55 C6 4E 4D 50 EB E4 17 A0 E9 86 16 FE C2 8F 61 U.NMP.....a
00000070  5B F6 52 7F 89 C1 7E 8E A8 A1 E5 1B 14 A2 18 B1 [.R...~.....
00000080  B8 05 A0 3B 78 FD 26 46 74 2C BD 65 0A EA 5E FA ...;x.&Ft,.e...^
00000090  5E 9A 1A A8 10 72 03 AF 5B 4E BD CB 0B B1 9F 8D ^.....r..[N.....
000000A0  4D 79 6B 34 74 9F 67 DC 67 DC 52 15 D6 6C 89 1F Myk4t.g.g.R..l..
000000B0  FD 46 CD 8A F5 9F 06 E5 AF CF 42 42 93 B9 7F 26 .F.....BB...&
000000C0  DF 55 50 72 5A 3A 90 B6 2A 6F 85 7F 67 C9 A9 49 .UPrZ:...*o..g..I
000000D0  40 7B 0A 76 A2 14 CE D1 A0 88 72 B4 72 24 DA 64 @{.v.....r.r$.d
000000E0  59 38 54 72 76 79 0A CF A8 CD FD 4A E4 F3 F8 B0 Y8Trvy.....J....
000000F0  5B 79 EF 26 5C 50 BF B3 56 3E 85 90 70 15 7E C1 [y.&P..V>...p..~
00000100  66 C3 2A A9 15 D4 F1 FF 68 D2 08 DF 4B A3 81 76 f.*.....h...K..v
00000110  17 CA 45 5C E6 73 0C 9A 01 47 61 A9 8E DB D0 98 ..E\.s...Ga.....
00000120  D4 5F 3B E6 2C 15 95 88 13 D1 57 7D A4 E1 D4 9E ._;.,.....W}....
00000130  C8 A2 CB 73 DB 68 9B 56 F2 FC 45 C3 A1 77 A6 2F ...s.h.V..E..w./
00000140  2C EC DC 9C D3 44 30 79 1C 09 11 70 8A 69 A1 F2 ,....D0y...p.i..
00000150  E1 47 0D B8 92 54 DD 17 9B 6C C9 C2 47 39 01 02 .G...T...l..G9..
00000160  DE A3 DC 91 84 33 62 01 12 47 CF EC 30 6A AC C1 .....3b..G..0j..
--- presentation.enc --0x0/0x40FF31--0%-----
```

Solution

so the concept is after i learned how the file gets encrypted by seeing its source code the idea is to do the reverse of what the program did so i just inverted each function and jumped "PDFENC" when i readed the file and also the metadata like

the size that existed after "PDFENC" that has 8 bytes so the real encrypted data start at index 14



and yes i used AI for the inverse of each function and for the main thanks to GCDxN7 community